



Faculté des Sciences et Techniques de Tanger

Université Abdelmalek Essaadi

---

Master's End-of-Study Project (PFE)  
Master in **[TODO: your master programme name]**

# Learning Offensive Navigation Policies on Heterogeneous Attack Graphs

A GNN Approach to Sequential Privilege Escalation  
in Active Directory

**Author:**  
Nizar Senbati

**Supervisor:**  
**[TODO: Supervisor name & title]**

---

Academic Year 2025 – 2026

# Acknowledgements

First and foremost, I would like to express my deepest gratitude to my academic supervisor GHADI Abderahim, for his unwavering guidance, patience, and invaluable feedback throughout this academic year.

I am equally grateful to my professional supervisor, ZAABOUL Jawad, and the entire team at NearSecure. Thank you for providing an environment that fosters innovation, for trusting me to pursue the intersection of offensive security and Deep Learning, and for the technical mentorship that bridged the gap between academic theory and real-world application.

A massive thank you is owed to the broader cybersecurity and open-source communities. The development of the GNN-AD-Navigator would not have been possible without the tireless work of the developers who build and maintain the tools we rely on. Specifically, I would like to acknowledge:

SpecterOps, for creating and open-sourcing BloodHound, which provided the foundational graph topology and data schema necessary for my model's ingestion pipeline.

Oliver Lyak ly4k, for developing Certipy, an essential tool that allowed me to accurately map complex Active Directory Certificate Services ADCS attack paths into my heterogeneous graph.

Mayfly Orange Cyberdefense, for creating GOAD Game of Active Directory. The immense complexity of this lab environment served as the perfect training ground for the neural network, allowing it to learn and discover emergent tactical behaviors.

I also want to thank Hack The Box Academy for their student program, which granted me accessible access to the inlanefreight Active Directory environment. This provided the critical cross-environment validation required to prove the model's efficacy outside of its training data.

Finally, a personal thank you to my father and family for the ongoing support, advice and guidance during the long hours spent debugging and studying. To everyone who supported me through this Master's journey: thank you.

# Abstract

Active Directory is a directory service so deeply embedded in corporate infrastructure that misconfigurations, no matter how small, can expose entire organisations to privilege escalation attacks. Tools like BloodHound are critical for both defenders and red teams: they map the environment as a graph and let operators query attack paths between any two objects. BloodHound’s built-in pathfinding, however, runs plain Dijkstra — it returns the shortest path, not necessarily the most operationally sound one, and it cannot learn from prior engagements.

This thesis asks whether a Graph Neural Network can learn a navigation policy from labelled attack traces and generalise it to entirely unseen enterprise environments. A Heterogeneous Graph Transformer is trained on a multi-domain lab environment and evaluated on a 4 000-node enterprise network it has never seen. The model recovers the expert-documented Domain Admin attack path without retraining, establishing cross-environment generalisation as a proof of concept. Identified architectural boundaries and a forward-looking hypothesis for a second-generation tool are discussed.

**Keywords:** Graph Neural Network, Heterogeneous Graph Transformer, Active Directory, Privilege Escalation, Attack Graph, Beam Search, Offensive Security, Cross-Environment Generalisation, Active Directory Certification Services.

# Résumé

Active Directory est un service d'annuaire si profondément intégré dans l'infrastructure des entreprises que la moindre erreur de configuration peut exposer une organisation entière à des attaques de privilege escalation. Des outils comme BloodHound sont essentiels à la fois pour les défenseurs et les red teams : ils cartographient l'environnement sous forme de graphe et permettent aux opérateurs de rechercher des chemins d'attaque entre deux objets. Cependant, le moteur de recherche de BloodHound utilise le simple algorithme de Dijkstra renvoie le chemin le plus court, et non le plus pertinent d'un point de vue opérationnel.

Cette thèse cherche à savoir si un Graph Neural Network peut apprendre une politique de navigation à partir de traces d'attaques étiquetées, et la généraliser à des environnements d'entreprise totalement inconnus. Un Heterogeneous Graph Transformer est entraîné sur un environnement de laboratoire multi-domaines, puis évalué sur un réseau d'entreprise de 4 000 nœuds qu'il n'a jamais vu auparavant. Le modèle parvient à retrouver le chemin d'attaque Domain Admin documenté par des experts, et ce sans réentraînement, validant ainsi la généralisation cross-environment comme un véritable Proof of Concept. Enfin, les limites architecturales identifiées ainsi que des pistes d'évolution pour un outil de seconde génération sont discutées.

**Mots-clés :** Réseau de neurones sur graphe, Transformateur de graphe hétérogène, Active Directory, Escalade de privilèges, Graphe d'attaque, Recherche en faisceau, Sécurité offensive, Généralisation inter-environnements.

# Contents

|                                                                 |            |
|-----------------------------------------------------------------|------------|
| <b>Acknowledgements</b>                                         | <b>i</b>   |
| <b>Abstract</b>                                                 | <b>ii</b>  |
| <b>Résumé</b>                                                   | <b>iii</b> |
| <b>List of Abbreviations</b>                                    | <b>ix</b>  |
| <b>General Introduction</b>                                     | <b>1</b>   |
| <b>1 Background and Related Work</b>                            | <b>4</b>   |
| 1.1 Active Directory: Architecture and Security Model . . . . . | 4          |
| 1.1.1 Core Objects and Structure . . . . .                      | 4          |
| 1.1.2 Authentication Protocols . . . . .                        | 5          |
| 1.1.3 Trust Relationships . . . . .                             | 5          |
| 1.2 Privilege Escalation in AD Environments . . . . .           | 5          |
| 1.2.1 Offensive ACL Abuse . . . . .                             | 5          |
| 1.2.2 DCSync Attack . . . . .                                   | 5          |
| 1.2.3 ADCS Attack Paths . . . . .                               | 5          |
| 1.2.4 SID History Abuse and RBCD . . . . .                      | 5          |
| 1.3 Existing Tools and Their Limitations . . . . .              | 5          |
| 1.3.1 BloodHound and SharpHound . . . . .                       | 5          |
| 1.3.2 bloodhound-python . . . . .                               | 6          |
| 1.3.3 Other Automated Attack Path Tools . . . . .               | 6          |
| 1.4 Graph Neural Networks . . . . .                             | 6          |
| 1.4.1 Message Passing and Neighbourhood Aggregation . . . . .   | 6          |
| 1.4.2 Graph Convolutional Network (GCN) . . . . .               | 6          |
| 1.4.3 Heterogeneous Graph Transformer (HGT) . . . . .           | 6          |
| 1.4.4 PyTorch Geometric and HeteroData . . . . .                | 6          |
| 1.5 Summary and Positioning . . . . .                           | 6          |
| <b>2 System Architecture and Design</b>                         | <b>7</b>   |
| 2.1 System Overview . . . . .                                   | 7          |
| 2.2 Data Collection and Preprocessing . . . . .                 | 7          |
| 2.2.1 Input Sources . . . . .                                   | 7          |

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| 2.2.2    | <code>clean_bloodhound.py</code> : Filtering and Multi-Domain Accumulation | 7         |
| 2.2.3    | <code>merger.py</code> : Normalisation and Merging                         | 7         |
| 2.2.4    | <code>stitcher.py</code> : ADCS Injection and Domain Detection             | 7         |
| 2.3      | Heterogeneous Graph Construction                                           | 7         |
| 2.3.1    | Node and Edge Type Schema                                                  | 7         |
| 2.3.2    | Direction Convention                                                       | 7         |
| 2.3.3    | Edge Extraction Details                                                    | 8         |
| 2.3.4    | PyG HeteroData Encoding                                                    | 8         |
| 2.4      | Navigation Model                                                           | 8         |
| 2.4.1    | HGT Encoder                                                                | 8         |
| 2.4.2    | Policy Head                                                                | 8         |
| 2.4.3    | Loss Function and Step Cost                                                | 8         |
| 2.5      | Beam Search Navigation                                                     | 8         |
| 2.5.1    | Algorithm                                                                  | 8         |
| 2.5.2    | Terminal State: DCSync                                                     | 8         |
| 2.5.3    | Cross-Environment Inference                                                | 8         |
| 2.6      | Audit Module                                                               | 8         |
| <b>3</b> | <b>Implementation</b>                                                      | <b>9</b>  |
| 3.1      | Technical Stack                                                            | 9         |
| 3.2      | Repository Structure                                                       | 9         |
| 3.3      | Data Pipeline                                                              | 9         |
| 3.3.1    | Script Responsibilities and Data Flow                                      | 9         |
| 3.3.2    | SharpHound Version Normalisation                                           | 9         |
| 3.4      | Training                                                                   | 9         |
| 3.4.1    | Training Data: GOAD                                                        | 9         |
| 3.4.2    | Kaggle Training Notebook                                                   | 9         |
| 3.4.3    | Model Checkpoints                                                          | 10        |
| 3.5      | Inference Pipeline                                                         | 10        |
| 3.6      | Graphical Interface                                                        | 10        |
| 3.6.1    | Design and Theme                                                           | 10        |
| 3.6.2    | Workflow Integration                                                       | 10        |
| 3.6.3    | Visualisation                                                              | 10        |
| <b>4</b> | <b>Experiments and Results</b>                                             | <b>11</b> |
| 4.1      | Experimental Setup                                                         | 11        |
| 4.1.1    | Training Environment: GOAD                                                 | 11        |
| 4.1.2    | Feature engineering                                                        | 11        |
| 4.1.3    | Test Environment: INLANEFREIGHT                                            | 12        |
| 4.1.4    | Test Environment: GOAD castelblack / kingslanding                          | 12        |

|          |                                                                                     |           |
|----------|-------------------------------------------------------------------------------------|-----------|
| 4.1.5    | Coverage Metric . . . . .                                                           | 12        |
| 4.2      | Result 1: wley $\rightarrow$ Domain Admin on INLANEFREIGHT . . . . .                | 12        |
| 4.3      | Result 2: Cross-Domain Trust Traversal . . . . .                                    | 12        |
| 4.4      | Result 3: Constrained Delegation — castelblack $\rightarrow$ kingslanding . . . . . | 12        |
| 4.5      | Result 4: Documented Limitation — Cross-Forest Foreign MemberOf . . . . .           | 12        |
| 4.6      | Discussion . . . . .                                                                | 13        |
| 4.6.1    | Model Coverage and Training-Set Ceiling . . . . .                                   | 13        |
| 4.6.2    | Expressiveness vs. Transferability: HGT vs. GCN . . . . .                           | 13        |
| 4.6.3    | Pipeline Completeness as a First-Class Contribution . . . . .                       | 13        |
| <b>5</b> | <b>Limitations and Future Work</b>                                                  | <b>14</b> |
| 5.1      | Data Collection Incompleteness . . . . .                                            | 14        |
| 5.2      | Expanding the Training Knowledge Base . . . . .                                     | 14        |
| 5.3      | Terminal State Learning via Offline Reinforcement Learning . . . . .                | 14        |
| 5.4      | Additional Future Directions . . . . .                                              | 14        |
| 5.4.1    | Calibrated Confidence Scoring . . . . .                                             | 14        |
| 5.4.2    | SharpHound Edge Vocabulary Versioning . . . . .                                     | 14        |
| 5.4.3    | Derived Attack Primitives . . . . .                                                 | 14        |
|          | <b>General Conclusion</b>                                                           | <b>15</b> |
| <b>A</b> | <b>Sanitised BloodHound JSON Example</b>                                            | <b>16</b> |
| <b>B</b> | <b>Heterogeneous Graph Schema</b>                                                   | <b>17</b> |
| <b>C</b> | <b>Key Code Excerpts</b>                                                            | <b>18</b> |
| <b>D</b> | <b>GUI Screenshots</b>                                                              | <b>19</b> |

# List of Figures

|                                                |   |
|------------------------------------------------|---|
| 1.1 Active Directory object hierarchy. . . . . | 4 |
|------------------------------------------------|---|

# List of Tables

# List of Abbreviations

**ADCS** Active Directory Certificate Services. 2

**DCSync** Directory Replication Service synchronisation attack. 2

**GOAD** Game of Active Directory. 2

**HGT** Heterogeneous Graph Transformer. 2

**PyG** PyTorch Geometric. 2

**RBCD** Resource-Based Constrained Delegation. 2

**SID** Security Identifier. 2

# General Introduction

## Context

Active Directory is the identity backbone of major enterprises. While typically isolated within the inside network, once the perimeter defenses are breached (e.g., via web service exploitation, phishing, or a compromised usb) an adversary gains the ability to directly interact and query the Active Directory Domain Controller. Through protocols such as LDAP and SMB the entirety of the organisation's IT infrastructure, data and services is essentially mapped out for the attacker. A vulnerable AD poses extreme levels of risk making hardening the AD network environment an extreme priority.

Once inside, attackers frequently leverage credential-based and ACL-based attacks such as pass-the-hash, DCSync, Kerberoasting are among the most impactful attack vectors in modern breaches. Red teams and security auditor alike rely on tools like BloodHound to map the attack surface and identify the vulnerable nodes during an engagement. However the analysis remains a highly manual process, time consuming, heavily dependent on human expertise and none scalable in large enterprise environments.

## Problem Statement

Bloodhound maps the entirety of the Active Directory to a graph database, each node represents an object, and the relationships between these objects is represented by edges. When a user runs a path finding query, BloodHound simply runs Dijkstra over the attack graph, it finds the shortest path, not necessarily the most realistic or discreet one.

The question this thesis asks : can a model learn a navigation policy and generalise that policy to an unseen enterprise environment?

## Objectives

The objective of this Thesis is to provide a proof of concept for learned attack path discovery over heterogeneous Active Directory graphs.

To that end this work aims to:

- Build an end-to-end pipeline transforming raw BloodHound and Certipy JSON to navigable [PyTorch Geometric \(PyG\) HeteroData](#) tensors.
- Train a GNN-based navigation policy and demonstrate cross-environment generalisation to unseen enterprise level environments.
- Identify and precisely describe the model’s limitations and architectural boundaries through systematic evaluation, establishing future work and improvements.

## Main Contributions

- A complete, modular heterogeneous graph pipeline from raw BloodHound/Certipy JSON to navigable [PyG HeteroData](#) tensors, covering [Active Directory Certificate Services \(ADCS\)](#), [Resource-Based Constrained Delegation \(RBCD\)](#), [GPLink](#), and [Security Identifier \(SID\)](#) history edges.
- A [Heterogeneous Graph Transformer \(HGT\)](#)-based navigation policy trained on [Game of Active Directory \(GOAD\)](#) traces and evaluated on the entirely unseen [INLANEFREIGHT](#) enterprise environment, demonstrating generalisation from a 351-node lab to a 4 000-node network.
- A diagnostic evaluation of the learned policy’s architectural boundaries identifying how each design oversight [Directory Replication Service synchronisation attack \(DCSync\)](#) goal-state blindness, computer-node edge directionality, and training-distribution sparsity propagated into observable failure modes, and estimating the performance ceiling imitation learning could reach if these constraints were lifted.
- A forward-looking system hypothesis describing a second-generation tool where identified oversights are addressed: a goal-conditioned policy with [DCSync](#)-aware terminal learning, bidirectional computer-node traversal, and an expanded training corpus outlining what the approach is capable of reaching beyond the bounds of this proof of concept.

## Report Outline

The remainder of this document is structured as follows. Chapter 1 establishes the theoretical foundation, reviewing the architecture of Active Directory, common privilege escalation techniques, the machine learning logic behind GNNs and the current state of graph-based attack path analysis. Chapter 2 details the methodology and system architecture, introducing the heterogeneous graph pipeline designed to transform raw BloodHound data into navigable tensors and present the core logic of the navigation

policy the loss function and the beam search method. Chapter 3 presents the architecture and training paradigm of the Graph Neural Network (GNN) navigation policy. Chapter 4 provides an empirical evaluation of the learned policy across multiple environments, documents both successful generalisations and precise failure modes, and identifies the architectural oversights that bound the current approach. Chapter 5 concludes the report by reviewing what this proof of concept achieved and laying out its limitations honestly. It then builds a hypothesis where a second generation tool, one where the identified oversights are properly handled, could theoretically reach as well as pinpointing where future research might target.

# Chapter 1

## Background and Related Work

### 1.1 Active Directory: Architecture and Security Model

#### 1.1.1 Core Objects and Structure

Active Directory (AD) is a directory service for Windows network environments. It is a distributed, hierarchical structure that allows for centralized management of an organization's resources, including users, computers, groups, network devices, file shares, group policies, servers and workstations, and trusts. AD provides authentication and authorization functions within a Windows domain environment.<sup>1</sup>

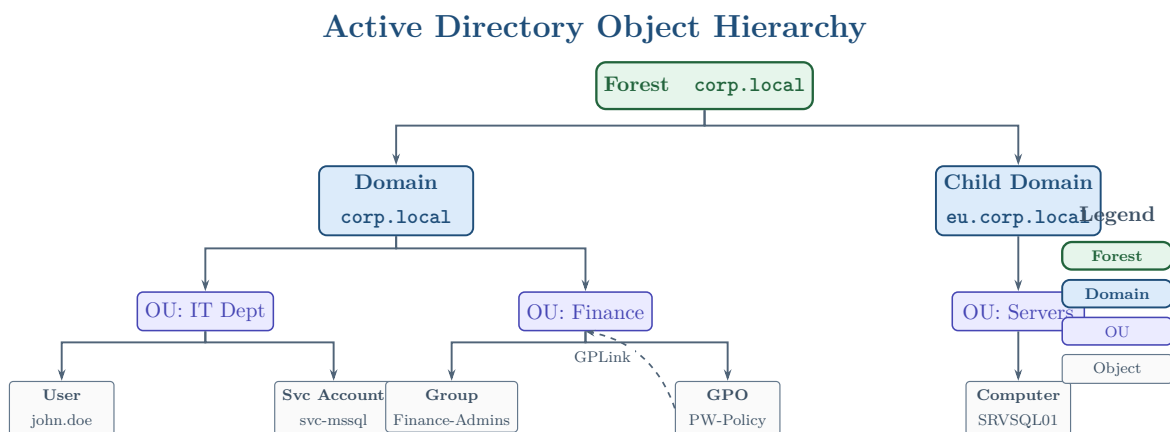


Figure 1.1: Active Directory object hierarchy.

A basic AD user account with no added privileges can be used to enumerate the majority of objects contained within AD, including but not limited to

- Domain Computers
- Domain Group Information
- Default Domain Policy

- Password Policy
- Domain Trusts
- Domain Users
- Organizational Units (OUs)
- Functional Domain Levels
- Group Policy Objects (GPOs)
- Access Control Lists (ACLs)

### 1.1.2 Authentication Protocols

[TODO: Write authentication section]

### 1.1.3 Trust Relationships

[TODO: Write trust relationships section]

## 1.2 Privilege Escalation in AD Environments

### 1.2.1 Offensive ACL Abuse

[TODO: Write ACL abuse section]

### 1.2.2 DCSync Attack

[TODO: Write DCSync section]

### 1.2.3 ADCS Attack Paths

[TODO: Write ADCS section]

### 1.2.4 SID History Abuse and RBCD

[TODO: Write SIDHistory and RBCD section]

## 1.3 Existing Tools and Their Limitations

### 1.3.1 BloodHound and SharpHound

[TODO: Write BloodHound/SharpHound section]

### 1.3.2 bloodhound-python

[TODO: Write bloodhound-python limitations]

### 1.3.3 Other Automated Attack Path Tools

[TODO: Write related tools survey]

## 1.4 Graph Neural Networks

### 1.4.1 Message Passing and Neighbourhood Aggregation

[TODO: Write GNN fundamentals]

### 1.4.2 Graph Convolutional Network (GCN)

[TODO: Write GCN section]

### 1.4.3 Heterogeneous Graph Transformer (HGT)

[TODO: Write HGT section]

### 1.4.4 PyTorch Geometric and HeteroData

[TODO: Write PyG section]

## 1.5 Summary and Positioning

While recent work has explored GNN-based modelling of attacker behaviour in AD graphs from a *defensive* perspective — approximating attacker decision-making to guide edge-blocking strategies **goel2025coevolutionary** — the complementary offensive direction remains largely unexplored. This thesis addresses that gap by learning an offensive navigation policy directly from labelled attack traces, evaluated on real enterprise environments rather than synthetic graphs. [TODO: Write summary and positioning]

# Chapter 2

## System Architecture and Design

### 2.1 System Overview

[TODO: Write system overview and include pipeline diagram]

### 2.2 Data Collection and Preprocessing

#### 2.2.1 Input Sources

[TODO: Write input sources section]

#### 2.2.2 `clean_bloodhound.py`: Filtering and Multi-Domain Accumulation

[TODO: Write `clean_bloodhound.py` section]

#### 2.2.3 `merger.py`: Normalisation and Merging

[TODO: Write `merger.py` section]

#### 2.2.4 `stitcher.py`: ADCS Injection and Domain Detection

[TODO: Write `stitcher.py` section]

### 2.3 Heterogeneous Graph Construction

#### 2.3.1 Node and Edge Type Schema

[TODO: Write node/edge schema — include a table]

#### 2.3.2 Direction Convention

[TODO: Write direction convention section]

### 2.3.3 Edge Extraction Details

[TODO: Write edge extraction details]

### 2.3.4 PyG HeteroData Encoding

[TODO: Write PyG encoding section]

## 2.4 Navigation Model

### 2.4.1 HGT Encoder

[TODO: Write HGT encoder section]

### 2.4.2 Policy Head

[TODO: Write policy head section]

### 2.4.3 Loss Function and Step Cost

[TODO: Write loss function section]

## 2.5 Beam Search Navigation

### 2.5.1 Algorithm

[TODO: Write beam search algorithm description — consider an Algorithm2e pseudocode block]

### 2.5.2 Terminal State: DCSync

[TODO: Write terminal state section]

### 2.5.3 Cross-Environment Inference

[TODO: Write cross-environment inference section]

## 2.6 Audit Module

[TODO: Write audit module section]

# Chapter 3

## Implementation

### 3.1 Technical Stack

[TODO: Write technical stack section — include a versions table]

### 3.2 Repository Structure

[TODO: Write repository structure section — include the directory tree as a listing]

### 3.3 Data Pipeline

#### 3.3.1 Script Responsibilities and Data Flow

[TODO: Write pipeline data flow section — include input/output table]

#### 3.3.2 SharpHound Version Normalisation

[TODO: Write SharpHound normalisation section]

### 3.4 Training

#### 3.4.1 Training Data: GOAD

[TODO: Write GOAD training data section]

#### 3.4.2 Kaggle Training Notebook

[TODO: Write Kaggle notebook section]

### 3.4.3 Model Checkpoints

[TODO: Write model checkpoints section]

## 3.5 Inference Pipeline

[TODO: Write inference pipeline section]

## 3.6 Graphical Interface

### 3.6.1 Design and Theme

[TODO: Write GUI design section]

### 3.6.2 Workflow Integration

[TODO: Write GUI workflow section]

### 3.6.3 Visualisation

[TODO: Write GUI visualisation section]

# Chapter 4

## Experiments and Results

### 4.1 Experimental Setup

#### 4.1.1 Training Environment: GOAD

Game of Active Directory, a deliberately vulnerable Active Directory lab, created for penetration testers to practice common infrastructure attacks and privilege escalation techniques. GOAD consists of three domains — `sevenkingdoms.local`, `north.sevenkingdoms.local`, and `essos.local` — interconnected through domain trust relationships, mimicking a realistic multi-domain enterprise architecture. The lab exposes various sorts of misconfigurations commonly found in enterprise environments, from unpatched system vulnerabilities to unprotected services and network misconfigurations. The attack paths goad provides are to be presented in a learnable and readable training format for the model. Each attack path is deconstructed into transitions, `current_node`, `next_hope`, `credentials_used`, `total_path_weight`, `train_on_step`, `notes` and the `step_cost`.

#### 4.1.2 Feature engineering

To a GNN model learning from a bloodhound scan, attack paths are of different importance, some are totally invisible while others are a gold mine for what the model needs to replicate. Paths that rely on unpatched vulnerabilities or outdated systems, for instance, leave no trace in the BloodHound graph and are marked with a `path_weight` of 0. Paths that are mostly visible with only two or less transitions that do not qualify have the latter marked through `train_on_step` set to zero. And at last, for an imitation learning model, given the choice between two valide transitions, the model choses based on training data, preferring the edge type it has seen most during training. To address this We are to more or less guide the model to gravitate towards the edge attack with less impact on the environment, the one with less noise and impact. To reach the desired effect each valid transition is scored. This score is reflected directly in the loss function through the `step_cost` feature.

In summary, the feature engineering phase introduced three mechanisms: filters

to eliminate CVE-dependent transitions. Weighted paths by a composite visibility score giving priority to transitions where the graph structure is the primary escalation vector. Finally a `step_cost` feature helping guide the model towards the stealthier less impactful edge in case of a choice presenting itself. This constitutes a simplified reward shaping approach consistent with the imitation learning literature.

### 4.1.3 Test Environment: INLANEFREIGHT

[TODO: Write INLANEFREIGHT setup section]

### 4.1.4 Test Environment: GOAD castelblack / kingslanding

[TODO: Write castelblack/kingslanding setup]

### 4.1.5 Coverage Metric

[TODO: Write coverage metric section — include a summary table of all queries]

## 4.2 Result 1: wley $\rightarrow$ Domain Admin on INLANE-FREIGHT

[TODO: Write Result 1 — include the key comparison figure]

## 4.3 Result 2: Cross-Domain Trust Traversal

[TODO: Write Result 2]

## 4.4 Result 3: Constrained Delegation — castelblack $\rightarrow$ kingslanding

[TODO: Write Result 3]

## 4.5 Result 4: Documented Limitation — Cross-Forest Foreign MemberOf

[TODO: Write Result 4]

## 4.6 Discussion

### 4.6.1 Model Coverage and Training-Set Ceiling

[TODO: Write coverage discussion]

### 4.6.2 Expressiveness vs. Transferability: HGT vs. GCN

[TODO: Write HGT vs. GCN discussion]

### 4.6.3 Pipeline Completeness as a First-Class Contribution

[TODO: Write pipeline completeness discussion]

# Chapter 5

## Limitations and Future Work

### 5.1 Data Collection Incompleteness

[TODO: Write data collection limitations section]

### 5.2 Expanding the Training Knowledge Base

[TODO: Write training knowledge base expansion section]

### 5.3 Terminal State Learning via Offline Reinforcement Learning

[TODO: Write offline RL future direction section]

### 5.4 Additional Future Directions

#### 5.4.1 Calibrated Confidence Scoring

[TODO: Write calibrated confidence section]

#### 5.4.2 SharpHound Edge Vocabulary Versioning

[TODO: Write edge vocabulary versioning section]

#### 5.4.3 Derived Attack Primitives

[TODO: Write derived primitives section]

# General Conclusion

[TODO: Write general conclusion ( 1 page)]

# Appendix A

## Sanitised BloodHound JSON Example

[TODO: Insert sanitised JSON excerpt as a lstlisting]

# Appendix B

## Heterogeneous Graph Schema

[TODO: Insert full graph schema table]

# Appendix C

## Key Code Excerpts

### Effective Weight Loss Function

```
1 # eff_weight combines path importance with step discretion cost
2 eff_weight = path_weight * (1 - step_cost)
```

Listing C.1: Effective weight computation in `prepare_training_examples.py`

[TODO: Add beam search loop and `has_dcsync_on` excerpts here]

# Appendix D

## GUI Screenshots

[TODO: Insert GUI screenshots]